8/7/25 AOA (Analysis of Algo).

- An algo is a finite set of instructions. if possible accomplished a particular task.

▸ Criteria.

1. Input:- zero or more quantites are extremally supplied
2. Output:- atleast one quantity is produced.
3. Definitness :- each instruction is clear & unambiguous
4. finitness :- algo. should terminate after a finite no. of steps.
5. effectiveness :- every instruction must be very basic so that it can be carried out.
6. Program :- Program is the expression of algo in programming language sometime such as function & subroutine are use simultaneously.

* Analysis of algo:-

1. Systematic education of how algo. behave concerning time, space complexity & practical efficiency while manufacturing diff types of data.

- What is important than performance.
  - modularity
  - correctness
  - maintainability
  - functionality
  - Robustness
  - userfriendlyness
  - simplicity
  - extensibility
  - relability.

- Asymtotic Notation

Worst case
- O (Big-O. notation)
- lowes bound

Average case
- Θ (Theta notation)
- Average.

Best case
- Ω (omega Notation)
- upper Bound.

8/7/25 - Analysis of Algorithm [AOA]

* Implementation of Insertion sort algorithm
* —||— Selection sort —||—
* —||— Merge sort —||—
* —||— Quick sort —||—
* WAP of implement Dijkstra's Algo. using greedy method.
* WAP of find min cost using Prim's & Kruskal Algo.
* —||— to find single source shortest path using Bellman ford —||— Algo by Dynamic Approach.
* WAP to implement n-queen problem using backtracking.
* Implementation of graph coloring using back tracking.
* —||— graph naive string matching —||—

- Time Complexity.

Intro:- It deal's with the quantification of the amt of time taken by set of code or algo. to process or run as a fun. of the amt of input.

defination:-

- Time complexity is define as the no. of times a Particular instruction set is executed rather than total time taken it depends on some extrnal factor's like compiler used some are the processor, speed etc.
- time taken T(P) taken by a program P is ~~so~~ sum of time taken compiler & the run time. (Execution).

1. constant time or O(1) :- is the time complexity of an algo. that always uses the same no. of op. regardless of no. of elements being operated on

O(1) does not implied that only one operation is used but rather than that the no. of operation is always constant.

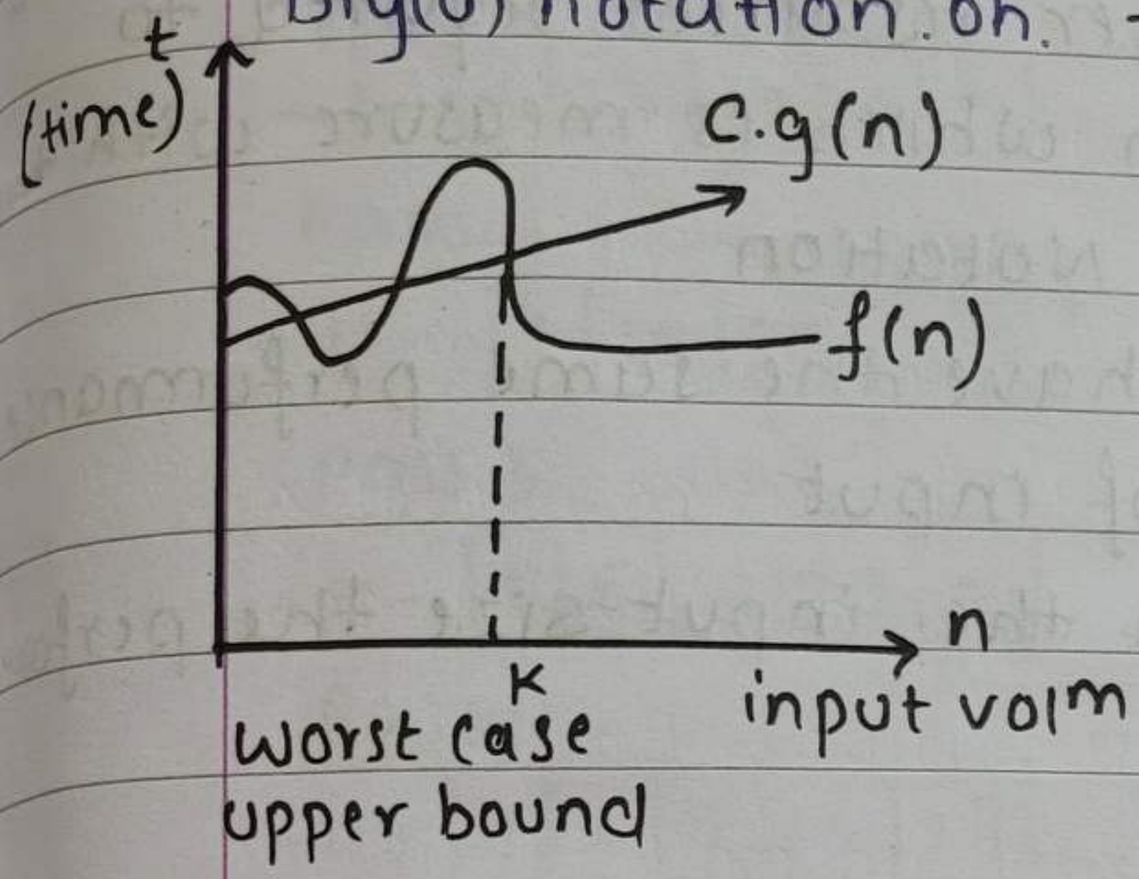
2. linear time or O(n) :- indicates that the time it takes to run an algo. grows in linear fashion. as n increases. in linear time searching a list of a thousand records should take roughly 10 times as long as searching. a list of 100 records.

3. Quadratic time or $O(n^2)$:- indicates that the time needed to run an algo. grows as square of n (n^2)

4. logarithmic time or O(logn) :- indicates that the time needed to run an algo. grows as logarithmic of n, for ex:- binary search on a sorted list is performed, the list is search by dividing it in half repetedly until to desire element is found the no. of division necessary to find the element grows with the algo. of n is based to rather than proprotionally took n. n (logn) is lower growth rate than O(n) these algo. have lower time complexity than linear time algo.

14/7/25 - Asymptotic Notation.

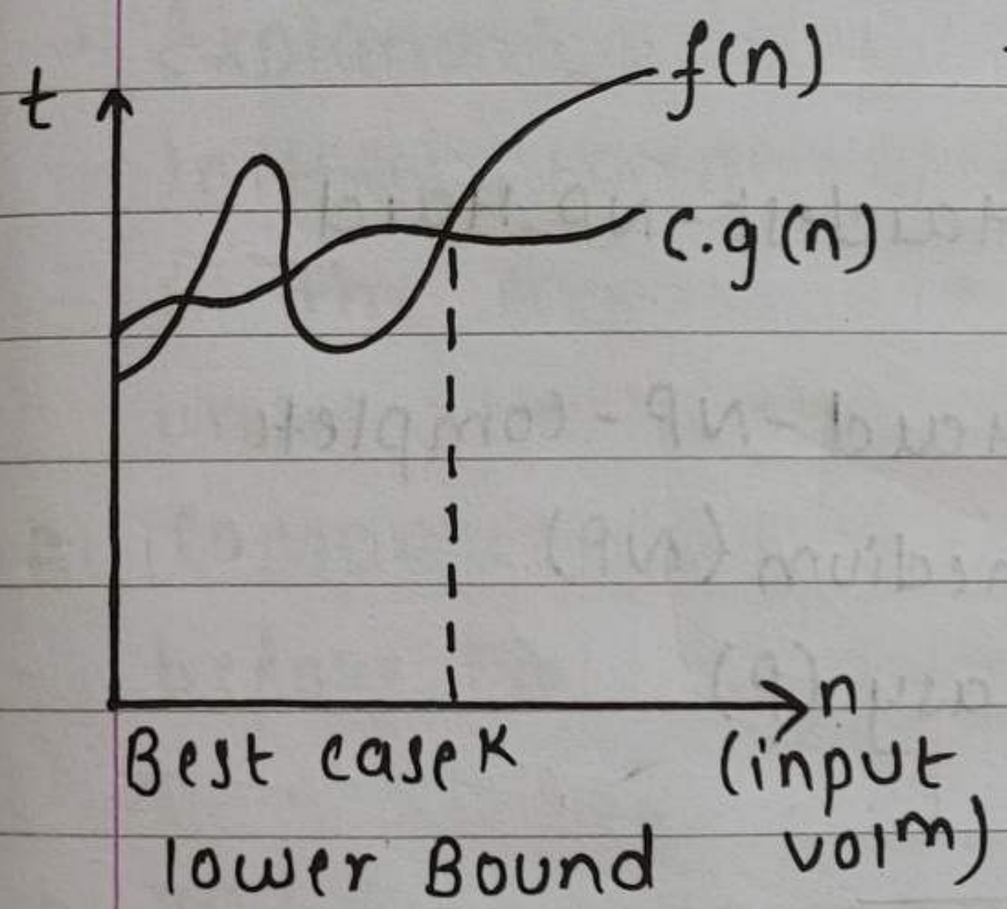
- Big(O) notation. oh. - maximum time required for execution $f(n) = O(g(n))$



$O(g(n)) = \{$ if there exist +ve constant c & K, such that there is.

$0 \le f(n) \le c * g(n)$ for all $n \ge K\}$

- Big omega (Ω)

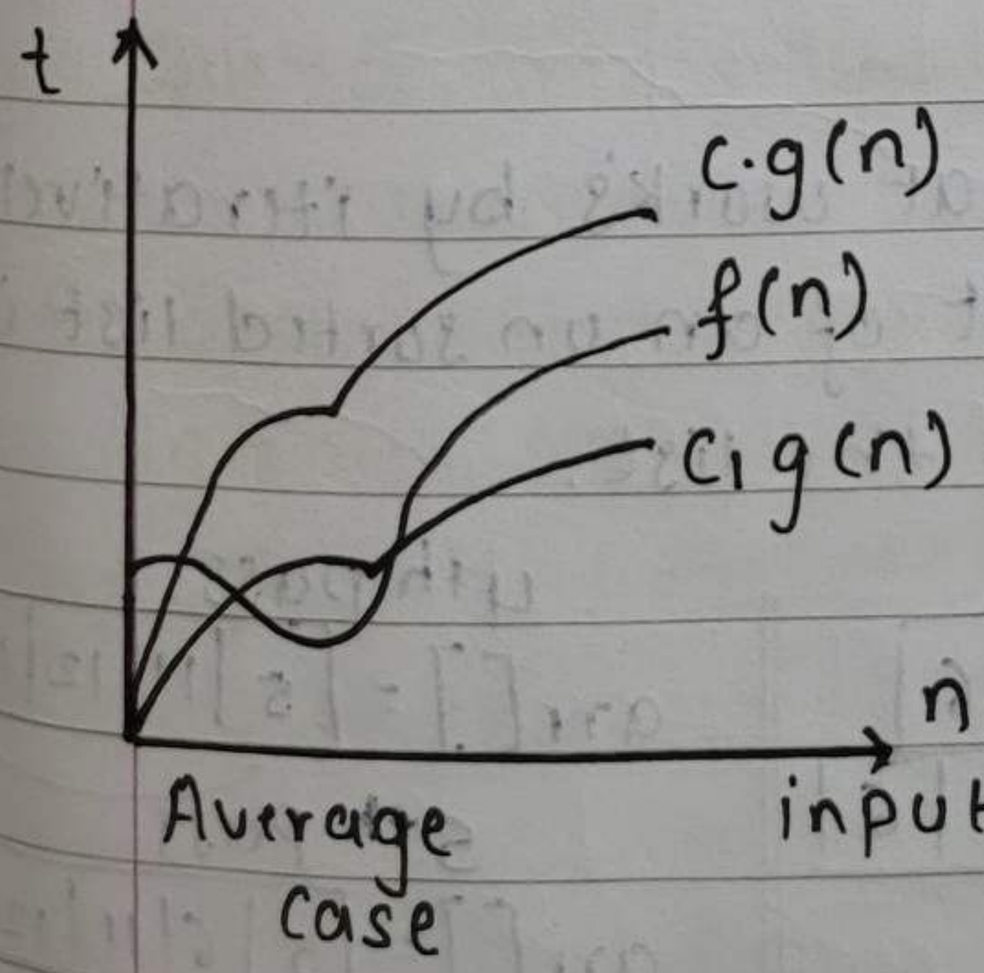


- minimum time required for execution $f(n) = \Omega(g(n))$

$\Omega(g(n)) = \{$ if there exist +ve constant c & K, such that there is.

$0 \le c * g(n) \le f(n)$ for all $n \ge K.\}$

- Theta (Θ)



- Average time required for execution $f(n) = \Theta(g(n))$

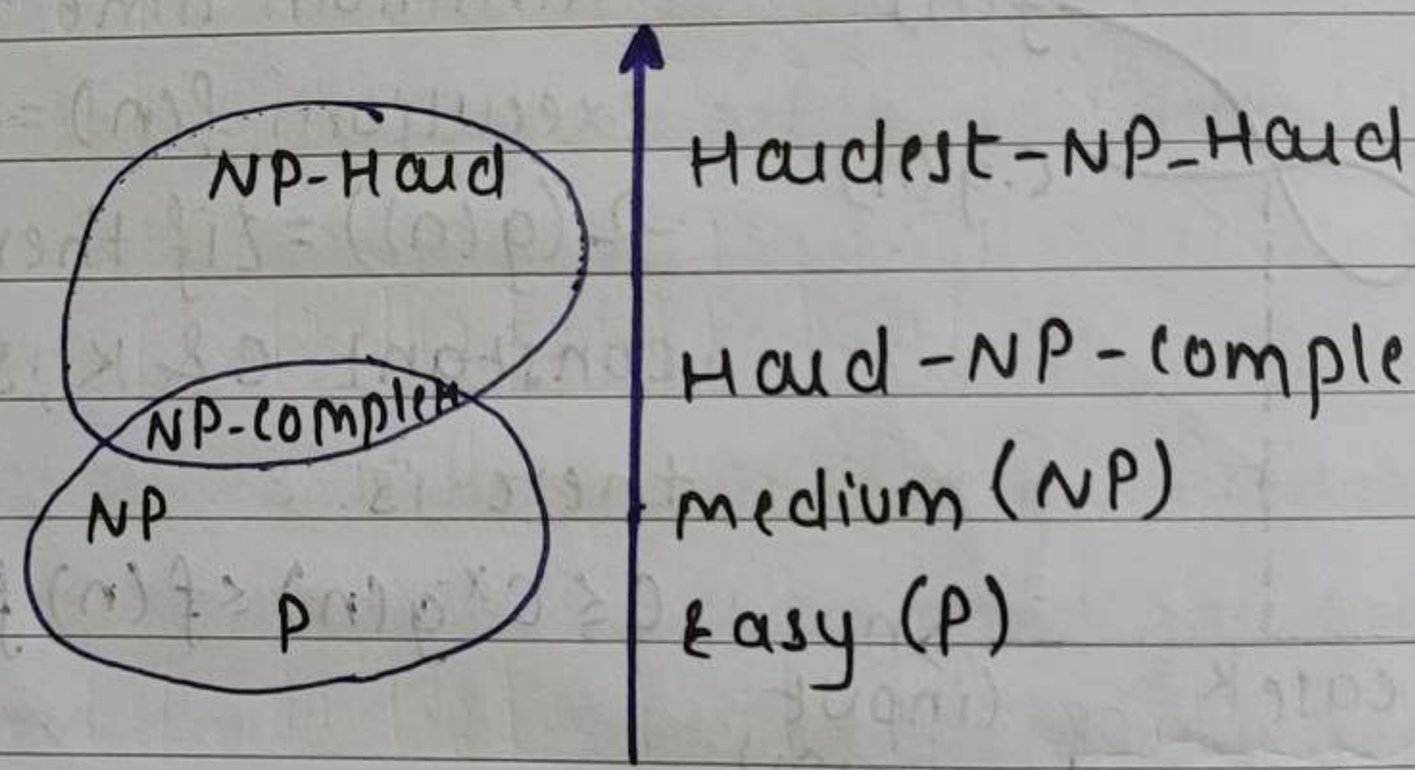
$\Theta(g(n)) = \{$ if there exist +ve constant c, c_1 & K, such that there is.

$0 \le cg(n) \le f(n) \le c_1 g(n)$ for all $n \ge K.\}$

—— o ——

- The efficiency of an algo depend's on amt of time, Storage & other resources required to execute the program which is measure with the help of asymptotic Notation.
- An algo may not have the same performance for different type of input
- with the increase in the input size the performance will change.

- P, NP, NP-complete, NP-hard.



—o—

16/7/25

- Insertion Sort

Simple sorting algo. that work's by iteratively inserting each element of an unsorted list into it's correct position of the list.

for Ex:-

arr[ ] = |12|11|13|5|6|

1st pass arr[ ] = |11|12|13|5|6|

2nd pass.

arr[ ] = |11|12|13|5|6|

3rd -11-

arr[ ] = |11|12|13|5|6|

4th pass

arr[ ] = |5|11|12|13|6|

5th pass

arr[ ] = |5|6|11|12|13|

- Initially : [23 | 1 | 10 | 5 | 2]

pass 1.

-11- [1 | 23 | 10 | 5 | 2]

pass 2.

-11- [1 | 10 | 23 | 5 | 2]

pass 3.

-11- [1 | 5 | 10 | 23 | 2]

pass 4.

-11- [1 | 2 | 5 | 10 | 23]

- Explanation :-

- Initial current element is 23, the 1st element in the array is assumed to be sorted part until zero index is : [23]. Now, the 1st pass :-

a. compare 1 with 23 since 1 is smaller insert 1 before 23, the sorted part until 1st index is [1,23]

b. 2nd pass compare 10 with 23 since 10 is greater than 1 & smaller than 23 insert 10 before 23 the sorted part until 2nd index is : [1, 10, 23].

c. 3rd pass, compare 5 with 1, 10 & 23 since 5 is greater than 1 & smaller than 10 insert 5 betn 1 & 10 the sorted part until 3rd index is :- [1, 5, 10, 23].

d. 4th pass compare 2 with 1, 5, 10, 23 since 2 is greater than 1 & smaller than 5 insert 2 betn 1 & 5 the sorted part until 4th index is [

e. Now the final array :- the sorted array is

1	2	5	10	23

$O(n)$ Best
$O(n^2)$ Worst
$O(n^2)$ Average
} Time complexity

—o—

21/7/25

- Selection Sort

swapping

arr[ ] = | 64 | 25 | 12 | 22 | 11 |

min element

swapping

arr[ ] = | 11 | 25 | 12 | 22 | 64 |

min element

sorted element.

swapping

arr[ ] = | 11 | 12 | 25 | 22 | 64 |

sorted element

minimum element

arr[ ] = | 11 | 12 | 22 | 25 | 64 | - sorted element.

—o—

Ex.2 Solve by selection sort.

arr[ ] = | 25 | 9 | 5 | 32 | 7 | 2 | 28 |

swapping

min element.

swapping

arr[ ] = | 2 | 9 | 5 | 32 | 7 | 27 | 28 |

sorted element

min element.

swapping element

arr[ ] = | 2 | 5 | 9 | 32 | 7 | 27 | 28 |

sorted element

min element

swapping element

arr[ ] = | 2 | 5 | 7 | 9 | 32 | 27 | 28 |

sorted

min element

arr[ ] = [2 | 5 | 7 | 9 | 27 | 32 | 28]

swapping element.

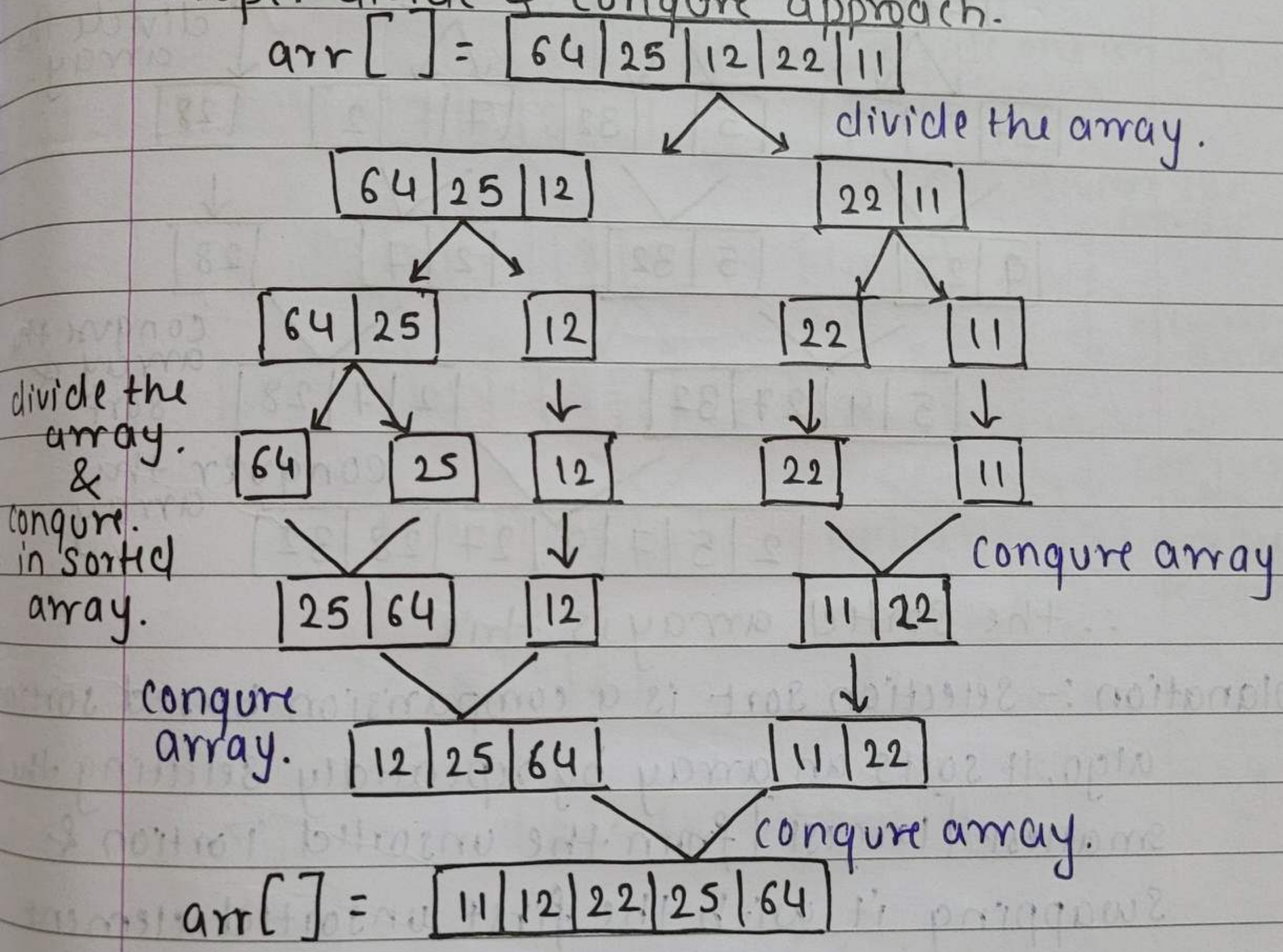
Sorted elem

↳ min element

arr[ ] = [2 | 5 | 7 | 9 | 27 | 28 | 32] - Sorted array.

- Merging Sort

Proper divide & conquer approach.

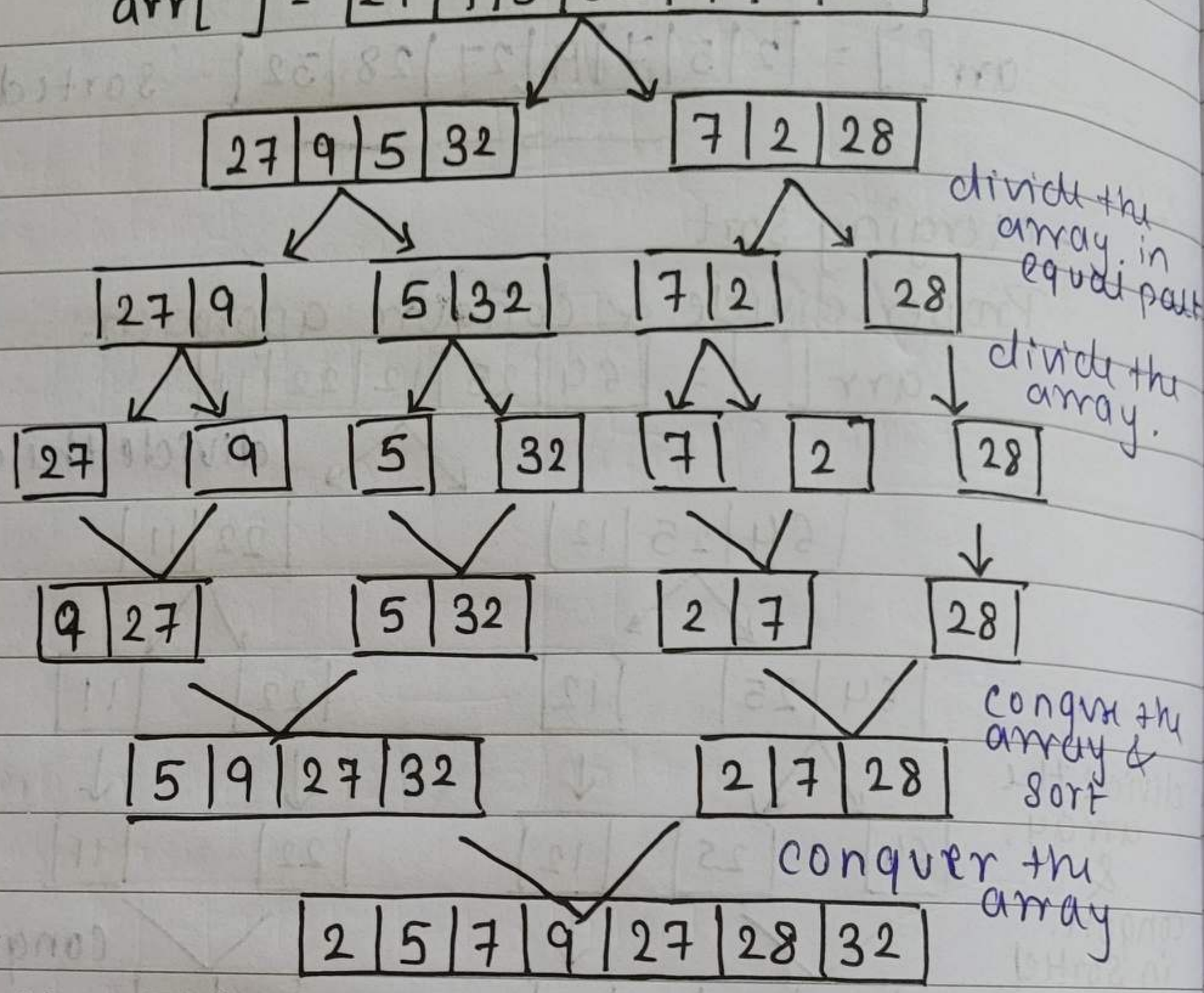


∴ this is the sorted array.

Ex2. Solve the following array with Merge Sort Method.

~~arr[ ] = |64|~~

arr[ ] = | 27 | 9 | 5 | 32 | 7 | 2 | 28 |



∴ the sorted array is this.

Explanation :- Selection sort is a comparision based sorting algo. it sorts an array by repeatedly selecting the smallest [largest] from the unsorted Portion & swapping it with the first unsorted element. this process continues, until the entire array is sorted.

Steps:- (1) first we find smallest element & compare it with first element if the first element is greater we swap the array this way we get smallest ~~amoung the remaining~~ element at its correct position.

(2) then we find the smallest amoung the remaining element [or second smallest] & swap it with second element

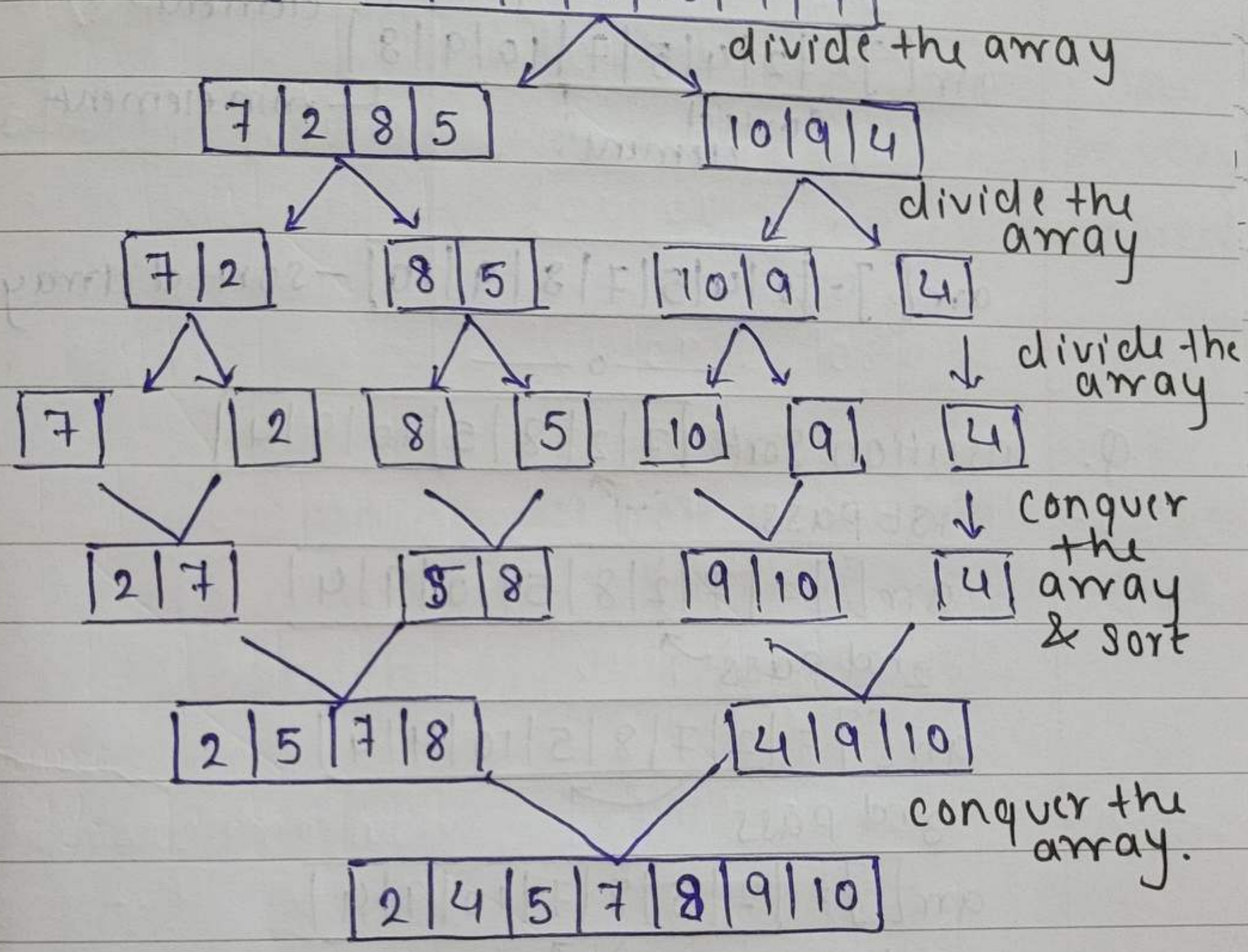
(3) we keep doing this until we get all element. moved to correct position.

—o—

25/7/25

Q. solve by Merge sort.

arr[ ] = | 7 | 2 | 8 | 5 | 10 | 9 | 4 |



∴ the sorted array is | 2 | 4 | 5 | 7 | 8 | 9 | 10 |

—o—

Q. Solve the following by selection sort.

arr[ ] = | 7 | 2 | 8 | 5 | 10 | 9 | 4 |

swapping element

arr[ ] = | 7 | 2 | 8 | 5 | 10 | 9 | 4 |

min element

swapping element

arr[ ] = | 2 | 7 | 8 | 5 | 10 | 9 | 4 |

min element

swapping element

arr[ ] = | 2 | 4 | 8 | 5 | 10 | 9 | 7 |

min element.

Sorted element

swapping element

arr[ ] = | 2 | 4 | 5 | 8 | 10 | 9 | 7 |

min element

Sorted element

swapping element

arr[ ] = | 2 | 4 | 5 | 7 | 10 | 9 | 8 |

min element

Sorted element

arr[ ] = | 2 | 4 | 5 | 7 | 8 | 9 | 10 | - sorted array.

—o—

Q. Insertion Sort | 7 | 2 | 8 | 5 | 10 | 9 | 4 |

1st pass

arr[ ] = | 7 | 2 | 8 | 5 | 10 | 9 | 4 |

2nd pass

arr[ ] = | 2 | 7 | 8 | 5 | 10 | 9 | 4 |

3rd pass

arr[ ] = | 2 | 5 | 8 | 7 | 10 | 9 | 4 |

4th pass

arr[ ] = | 2 | 5 | 7 | 8 | 10 | 9 | 4 |

5th pass

arr[ ] = | 2 | 5 | 7 | 8 | 9 | 10 | 4 |

6th pass

arr[ ] = | 2 | 5 | 7 | 8 | 9 | ~~10~~ 4 | 10 |

7th pass

Solve all -||-

arr[ ] = | 2 | 4 | 5 | 7 | 8 | 9 | 10 |

sorted array.

Page No.:
Date:

25/7/25

- Quick Sort

Quick sort is a sorting algorithm based on quick divide & conquer approach that picks an element as a pivot & partitions that given array around the picked pivot in it's correct position in a sorted array.

working :-

- choose a pivot - select an element from the array as a pivot the choice of pivot can vary
  eg:- 1st element, last element, random element middle. ~~last~~ element)
- Partition the array :-
- (rearranged the array) - around the pivot after partitioning all element's smaller than the pivot will be on it's left, & all elements greater than the pivot will be on it's right. the pivot is then in it's correct position, & we obtain the index of the pivot.
- recursively call - recursively call the same process to the partition, sub-arrays (left of the pivot).

eg:-

0	1	2	3	4
10	80	30	90	40

Step1:- pivot selection, the last arr[4] = 40 is choosen as the pivot

initial pointers = i = -1 & j = 0

arr[ ] = | 10 | 80 | 30 | 90 | 40 |

compare all with pivot.

Step I - arr[] = |10|80|30|90|40|

Step II - |10|80|30|90|40|

|10|80|30|90|40|

Step III - |10|80|30|90|40|

|10|80|30|90|40|

Step IV - |10|80|30|90|40|

|10|30|80|90|40|

Step V - |10|30|80|90|40|

|10|30|80|90|40|

Step VI - |10|30|80|90|40| → now compare with Pivot 40.

Step VII - |10|30|40|90|80|

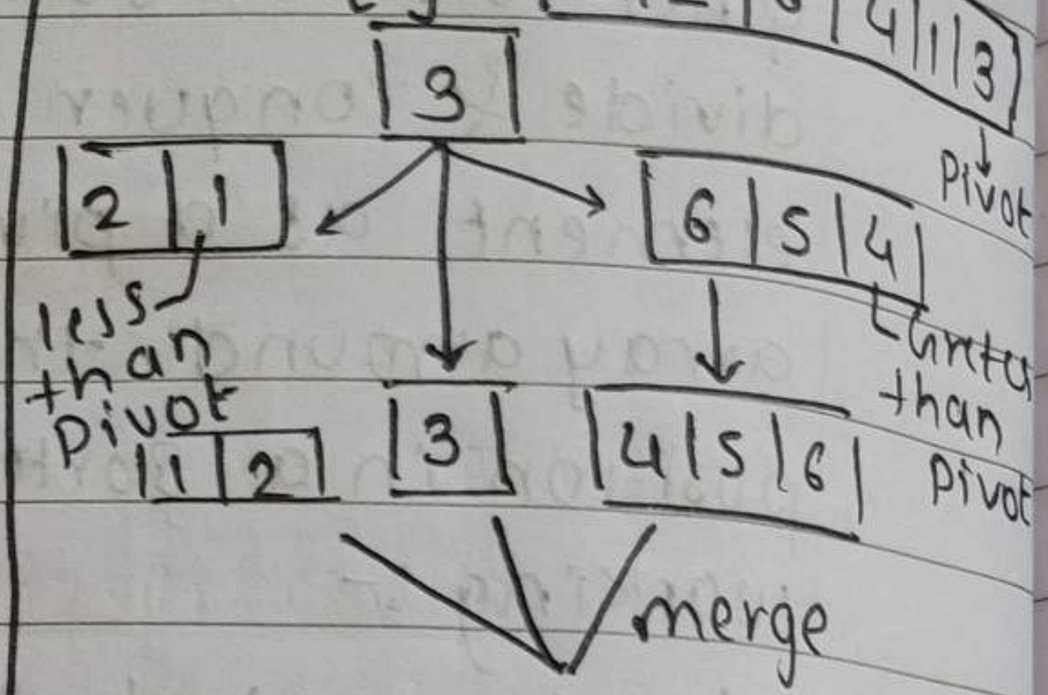
|10|30| |80|90|

|10|30|40|80|90|

as 90 is greater than 80 & 80 is ~~greater~~ smaller than 90 Hence, 80 is placed betn 40 & 90.

23/7/25

arr[] = |5|2|6|4|1|3|



arr[] = |1|2|3|4|5|6|